Redictado Orientación a Objetos 1 - 2026 - Parcial fecha 1

Comentarios generales:
En cada hoja, escriba apellido, nombre, legajo o DNI, y número de hoja / total de hojas entregadas.
Debe realizar todos los ejercicios para aprobar. No deje ninguno en blanco.
Debe entregar al menos una hoja con sus datos, aunque no haya completado todo lo pedido.

Ejercicio 1

Se desea desarrollar un sistema de gestión para una biblioteca pública. La biblioteca posee colecciones de materiales bibliográficos, que incluyen libros y revistas. De cada material se conoce su título y año de publicación. Los libros tienen además un autor y un ISBN. Las revistas tienen un número de edición.

Los socios pueden obtener préstamos de materiales. Cada préstamo registra el socio, el material prestado, la fecha de préstamo y la fecha de devolución pactada. Un material puede haberse prestado múltiples veces a lo largo del tiempo. Los socios tienen un número de socio que los identifica, nombre y dirección de correo electrónico.

El sistema debe permitir las siguientes funcionalidades:

- ***Registrar un préstamo:*** *dado un socio y un material, se registra el préstamo con la fecha actual. Solo se puede prestar si el material no está actualmente en préstamo activo.*
- ***Registrar la devolución:*** *dado un préstamo, se registra la fecha efectiva de devolución.*
- **Listar materiales prestados a un socio:** dado un socio, retorna todos los materiales que tiene actualmente en préstamo (sin devolución registrada).
- **Listar préstamos vencidos:** retorna todos los préstamos cuya fecha de devolución pactada ya pasó y que no han sido devueltos, ordenados por fecha de préstamo.
- **Calcular el historial de un socio:** retorna todos los préstamos que realizó un socio, ordenados cronológicamente por fecha de préstamo.

Tareas:

1. Realice un diagrama de clases UML donde identifique las clases del dominio, atributos, y las relaciones con sus roles y cardinalidades según corresponda. No debe incluir comportamiento.

NOTA: Asegúrese de que su modelo permite resolver todas las funcionalidades pedidas.

Ejercicio 2

Se tiene un sistema que calcula el costo de envíos de paquetes. Actualmente existen dos formas de envío: EnvioEstandar y EnvioExpress. La clase Paquete mantiene directamente la lógica de cálculo usando condicionales, como se muestra:

```
public class Paquete {
    private double peso;        // en kg
    private double volumen;     // en cm³
    private double distancia;   // en km
    ptivate String tipoEnvio;  // "estandar" o "express"

    public Paqcete(double peso, double volumen, double distancia, String
tipoEnvio) {
        this.peso = peso;
        this.volumen = volumen;
        this.distancia = distancia;
        this.tipoEnvio = tipoEnvio;
    }

    public double calcularCosto() {
        if (this.tipoEnvio.equals("estandar")) {
            return this.peso * 150 + this.volumen * 0.5;
        } else if (this.tipoEnvio.equals("express")) {
            return this.peso * 400 + this.volumen * 1.2 + this.distancia
* 50;
        }
        return 0;
    }}
```

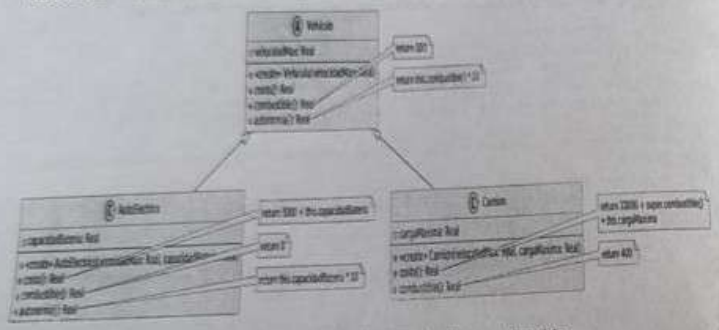
Se desea agregar un nuevo tipo de envío internacional, que toma como base el costo estándar calculado, le aplica un recargo del 80%, y suma un arancel fijo de $2000. Además, se requiere poder cambiar el tipo de envío de un paquete en tiempo de ejecución.

Tareas:

1. Rediseñe lo necesario para desacoplar la clase Paquete de la lógica de cada tipo de envío, de forma que el tipo de envío pueda cambiarse dinámicamente. Delegue polimórficamente el cálculo del costo en el tipo de envío asignado. Realice un diagrama de clases UML completo de su diseño (incluyendo métodos y constructores), incorporando también el nuevo tipo de envío internacional.
2. Reimplemente en Java respetando el nuevo diseño. Implemente todos los métodos necesarios.
3. Instancie un paquete de 3 kg, 500 cm³ a enviar a 200km. Asigne envío estándar, calcule el costo y guárdelo en una variable. Luego, cambie el tipo de envío a express y actualice el costo.

Ejercicio 3

Dado el siguiente modelo y los fragmentos de código, complete el cuadro con los valores correspondientes al resultado de evaluar cada sentencia.



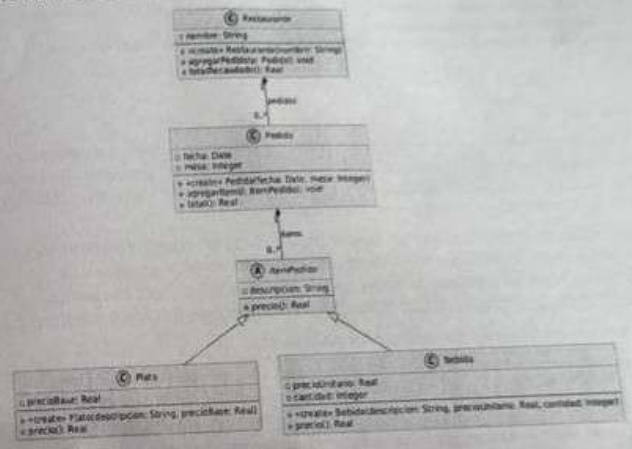
Dada la siguiente instanciación, complete los resultados en la tabla

```
AutoElectrico ae = new AutoElectrico(200, 80);
Camion cam = new Camion(120, 5);
```

	Sentencia a evaluar	Resultado
3.1	ae.costo();	5000 + 80 = 5080
3.2	ae.autonomia();	80 * 10 = 800
3.3	cam.combustible();	400
3.4	cam.costo();	80000 + 100 * 5 = 80 500
3.5	cam.autonomia();	400 * 10 = 4000

Ejercicio 4

El diagrama de clases que se muestra a continuación documenta parte del diseño de un sistema de gestión de pedidos de un restaurante.



Su funcionamiento es el siguiente:

- Pedido.total(): retorna la suma de los precios de todos sus ítems.
- Plato.precio(): retorna el precio base.
- Bebida.precio(): retorna el precio unitario multiplicado por la cantidad.
- Restaurante.totalRecaudado(): retorna la suma del total de todos los pedidos.

Tareas:

1. Respetando las firmas definidas, implemente los constructores de las clases Restaurante, Pedido, Plato y Bebida.
2. Implemente los métodos total() de Pedido, precio() de Plato, precio() de Bebida y totalRecaudado() de Restaurante. Utilice Streams cuando resulte apropiado.
3. ¿Es posible agregar un nuevo tipo de ítem (por ejemplo, Postre) sin modificar la clase Pedido? Justifique brevemente.